

Phase Three: Data Operations and Query Specification

Group 9

Operational Processes and Business Rules

The system supports the workflow of project management, starting from client onboarding and continuing through execution, monitoring, and evaluation.

When a new client begins working with the company, their basic information, including company name, contact person, email, phone number, address, and industry, is stored in the system. After registration, a project manager creates a project associated with that client. Each project is linked to exactly one client, while a client may have multiple projects.

During project creation, attributes such as budget, start and end dates, priority level, and project status are defined. The system performs validation checks to ensure that the budget is positive and that the start date is earlier than the end date. The system defines standardized value sets for multiple attributes to ensure consistency and data integrity across the database:

- **Expense payment status:** Pending, Paid, Cancelled, Not Paid
- **Risk status:** Open, Mitigated, Closed
- **Milestone status:** Not Started, In Progress, Pending, Completed, Delayed
- **Task status:** Not Started, In Progress, Completed, Blocked
- **Employee status:** Active, Remote, On Leave
- **Project status:** Planning, Active, Completed, On Hold, Cancelled
- **Project priority:** Low, Medium, High, Critical

Task and Milestone Management

Tasks are created as individual work items and are connected to milestones through the **Milestone_Tasks** relationship. This means the relationship between tasks and milestones is **many-to-many**: one milestone may include many tasks, and one task may be connected to more than one milestone when it supports multiple project stages.

Milestones belong to projects and represent important stages of project progress. Each milestone has a due date, completion percentage, and status. Milestone status values include **Not Started**, **In Progress**, **Pending**, **Completed**, and **Delayed**.

Tasks are assigned to employees through the **Task_Assignments** relation. This also creates a **many-to-many** relationship: one task can be assigned to multiple employees, and one employee can work on multiple tasks. Each task has estimated hours and a status. Task status values include **Not Started**, **In Progress**, **Completed**, and **Blocked**.

Time logs are recorded separately and connect employees with the tasks they worked on. This allows the system to track actual working hours, compare them with estimated hours, and support billing and workload analysis.

Skills and Role Requirements

The system manages employee competencies using a skills framework. Employees possess skills with defined proficiency levels, while roles define required skills and expected proficiency levels.

This allows the system to support analysis of employee capabilities and identify gaps between employee skills and role requirements, which can be used for training and workforce planning.

Time Logging and Billing

Employees record the hours they spend working on tasks in the Time_Logs table, which is linked to both employees and tasks.

The system enforces several constraints to maintain realistic data:

- Time logs cannot be added to tasks that are already completed or approved
- Total logged hours for a task should not exceed 1.5 times the estimated hours unless special approval is given

Billable amounts are calculated based on the employee's hourly rate and the number of hours logged.

Project Expenses

The Project_Expenses table tracks financial activity related to projects. Each expense is linked to a single project and includes information about its amount, expense date, and payment status.

The system monitors total project spending and compares it with the project budget. When a new expense is added, it checks whether the total would exceed the budget. If so, a warning is triggered or the operation is restricted.

Risk Management

The system includes risk management functionality. Project managers can define risks by assigning probability and impact values. Based on these inputs, a risk score is calculated.

Risks are categorized as *Low*, *Medium*, *High*, or *Critical*, and they may have statuses such as *Identified*, *Analyzed*, *Mitigated*, *Occurred*, or *Closed*.

Projects with high risk scores require more frequent monitoring and updates.

Organizational Structure

The system models organizational relationships between departments, roles, and employees.

- Departments group employees within the organization
- Roles define job responsibilities and are associated with departments
- Employees are assigned roles and belong to departments

This structure supports organizational analysis and workforce management.

Reporting and Analytical Tasks

The system supports reporting and analytical operations to help management monitor performance and make decisions.

- **Monthly Billable Hours Report**
 - Calculates total billable hours for each employee within a month using time logs marked as billable. Results are sorted for performance evaluation.
- **Skill Demand Analysis**
 - Analyzes which skills are most frequently required, helping identify training and hiring needs.
- **Project Profitability Report**
 - Compares revenue (from billable time logs) with project expenses to determine profitability.
- **Department Utilization Report**
 - Measures how efficiently departments use available working hours based on logged time.
- **Risk Exposure Dashboard**
 - Groups projects by priority and calculates total risk exposure based on risk scores and project budgets.

Data Volume

- Clients: 500 rows
- Projects: 900 rows
- Milestones: 2,700 rows
- Tasks: 9,000 rows
- Milestone_Tasks: 9,000 rows
- Task_Assignments: 12,000 rows
- Time_Logs: 10,000 rows
- Project_Expenses: 900 rows
- Project_Risks: 1,800 rows
- Project_Employee: 3,000 rows

- Employees: 700 rows
- Departments: 35 rows
- Roles: 105 rows
- Department_Roles: 105 rows
- Skills: 300 rows
- Employee_Skills: 1,500 rows
- Role_Skills: 500 rows

Total: 53,045 records

Data Collection Method

The dataset is generated using Python Faker for realistic data such as names, contact information and descriptions.

Query Specification

The system includes a set of queries that support both operational processes and analytical reporting. These queries are designed based on real business needs and reflect how the system would be used in practice.

- **Query 1: Employee availability check before task assignment**

This query is used before assigning a task to ensure that the selected employee has enough capacity. The system counts the number of active tasks currently assigned to each employee and filters those whose workload is below a certain threshold (for example, fewer than five active tasks). This query requires INNER JOINS between the Employees, Task_Assignments, and Tasks tables, along with aggregation.

- **Query 2: Project budget vs. actual expenses comparison**

This query is used to monitor project spending by comparing the project budget with the total recorded expenses. It calculates the sum of all expenses per project and shows how much of the budget has already been used and how much remains. This helps managers see whether the budget has been reached or is close to the limit. The query involves a LEFT JOIN between Projects and Project_Expenses and uses aggregation.

- **Query 3: Client project portfolio summary**

This query provides an overview of clients and their projects. It retrieves all clients along with their active projects, total budgets, and average completion percentages based on milestones. The query involves multiple joins between Clients, Projects, and Milestones and includes aggregation.

- **Query 4: Project profitability analysis**

This query calculates profit for each project by comparing total revenue and total expenses. Revenue is computed from billable time logs, where hours are multiplied by employee hourly rate. Expenses are taken from the Project_Expenses table and represent the total amount spent on the project. The query helps determine whether a project is profitable or not. It involves joins between Projects, Time_Logs, Employees, and Project_Expenses, and may include subqueries.

- **Query 5: Risk exposure by project priority**

This query calculates total risk exposure grouped by project priority. It computes a risk score based on probability and impact level, combines it with project budgets, and aggregates results per priority category. The query involves joins between Projects and Project_Risks and may use derived tables or subqueries

- **Query 6: Employee Skill Coverage per Department**

This query calculates the total number of certified skills and the average proficiency level for employees within each department. It aggregates competency data by department to highlight which teams possess the strongest verified skill sets. This helps HR and department heads identify high-performing units and allocate targeted training or cross-departmental resources effectively. The query involves INNER JOINS between Departments and Employees, a LEFT JOIN with Employee_Skills filtered on certification status, and uses COUNT and AVG aggregations.

- **Query 7: Role Skill Requirements vs Employee Skill Coverage**

This query compares the required proficiency for each role-skill combination against the actual number of employees who meet or exceed that threshold. It reveals which roles may be understaffed in specific technical or functional competencies. This is valuable for strategic recruitment planning and identifying where upskilling initiatives are most urgently needed. The query involves JOINS

between Roles, Role_Skills, and Skills, with a LEFT JOIN to Employee_Skills filtered by proficiency level, using COUNT and GROUP BY operations.

- **Query 8: Employee Skill Gap vs Role Requirements**

This query identifies employees whose current skill proficiency falls below the required level for their assigned role, or who lack the required skills entirely. It highlights specific competency mismatches at the individual level to support personalized development plans and performance coaching. The query involves JOINS between Employees, Roles, and Role_Skills, with a LEFT JOIN to Employee_Skills to detect missing or insufficient proficiencies, using conditional logic and filtering.

- **Query 9: Overdue Milestones with Task Completion Status**

This query detects milestones that have passed their due date and are not yet marked as completed or achieved. It also calculates the percentage of associated tasks that have been finished, providing insight into project bottlenecks and execution delays. This helps project managers prioritize interventions, adjust timelines, and communicate realistic delivery expectations to stakeholders. The query involves JOINS between Milestones, Milestone_Tasks, and Tasks, with date filtering and conditional aggregation for task status.

- **Query 10: Employee Workload vs Estimated Hours**

This query compares the total hours logged by employees against the originally estimated hours for their assigned tasks. It calculates the absolute variance and percentage difference to identify employees who are overworking or underutilized relative to task scope. This supports capacity planning, workload balancing, and estimation accuracy reviews without relying on date-based filters. The query involves JOINS between Employees, Time_Logs, and Tasks, using SUM, COALESCE, and variance calculations with GROUP BY.

- **Query 11: Employee-to-Time-Log Completeness View**

This query analyzes how actively each employee is logging their working hours in the system. It calculates the total number of time logs and total hours per employee and classifies their activity level as “No Activity”, “Low Activity”, or

“Active”. It helps management identify employees who are not properly tracking their work or are highly engaged in time logging.

- **Query 12: Employee Peer Comparison**

This query compares employees within the same department using a self-join on the Employees table. It generates pairs of employees who belong to the same department, allowing analysis of team structure and internal peer relationships. It is useful for understanding collaboration opportunities and department-level organization.

- **Query 13: Salary Benchmarking by Department**

This query compares each employee’s hourly rate with the average hourly rate of their department. It classifies employees as earning “Above Average”, “Average”, or “Below Average” salaries relative to their peers. The query supports compensation analysis, salary benchmarking, and HR decision-making.

Triggers

- **Trigger 1: `check_budget_before_expense`**

This trigger function checks whether adding a new project expense would exceed the allocated project budget before the expense is inserted into the database. It retrieves the project’s total budget and calculates the current total expenses for that project. If the new expense would cause the project to go over budget, the trigger raises an exception and prevents the insertion. This helps maintain financial control and ensures that project spending remains within approved limits.

- **Trigger 2: `block_time_on_finished_tasks`**

This trigger is executed before inserting a new record into the Time_Logs table. It checks the current status of the associated task in the Tasks table. If the task is already marked as 'Completed' or 'Blocked', the insertion is rejected with an error message. This prevents inaccurate time tracking on finished or halted tasks and ensures that employee workload and billing records remain consistent with the actual project status.

- **Trigger 3: task_hour_limit**

This trigger is executed after inserting or updating a record in the Time_Logs table. It automatically calculates the total logged hours for the associated task and compares the cumulative total against the task's estimated hours. If the sum exceeds 1.5 times the original estimate, the operation is rejected with a clear error message. This enforces workload discipline, prevents uncontrolled time tracking on individual tasks, and ensures that project planning remains aligned with actual resource consumption.

Indexes

An index on **Task_Assignments(employee_id, task_id)** helps in checking employee assignments and retrieving tasks for a specific employee more efficiently. This is especially useful for queries that verify whether an employee is already assigned to a task or count how many tasks they currently have, which are common operations in task allocation.

An index on **Projects(status)** for active projects improves performance of queries that focus only on ongoing projects. Since many reports and monitoring operations work only with active projects, indexing only those rows reduces unnecessary scanning of completed or inactive records.

An index on **foreign key columns** such as project_id, task_id, and employee_id is used to improve the performance of queries that involve joining related tables. Since many queries in the system retrieve data by linking tables such as Projects, Tasks, and Employees, these indexes help the database match rows more efficiently. This reduces the need to scan entire tables and improves the performance of join operations.

Functions

- **Function 1: calculate_project_completion**

This function calculates the overall completion percentage of a specific project by computing a weighted average of its milestones. It retrieves all milestones linked to the project, multiplies each milestone's completion_percentage by the number of tasks it contains, and divides the sum by the total number of tasks across all milestones. This weighted approach ensures that larger, more complex milestones contribute proportionally more to the final project progress score. The function returns a numeric value between 0 and 100, or 0 if no tasks exist for the project. It is primarily used in executive dashboards and project status tracking to provide a precise, workload-adjusted measure of project advancement.

- **Function 2: get_task_time_efficiency**

This function evaluates how efficiently a specific task is being executed by comparing the total hours logged against its originally estimated hours. It calculates the ratio of actual to estimated time and maps the result to a clear business category: 'Efficient' ($\leq 80\%$), 'On Track' (80–100%), 'Over Estimate' (100–150%), or 'Critical' ($> 150\%$). If the task lacks an estimated time, it safely returns 'No Estimate'. This function supports project monitoring and resource planning by quickly highlighting tasks that deviate from initial plans, enabling managers to adjust timelines, review estimation practices, or intervene before delays impact project delivery. It directly aligns with the system's $1.5\times$ hour cap business rule.

- **Function 3: get_employee_workload_status**

This function assesses an employee's current workload capacity by counting the number of active tasks ('Not Started' or 'In Progress') currently assigned to them. Based on predefined organizational thresholds, it returns a clear status indicator: 'Available' (fewer than 5 active tasks), 'At Capacity' (5–8 tasks), or 'Overloaded' (more than 8 tasks). This supports proactive resource management and task assignment workflows by ensuring that new work is only allocated to employees with sufficient bandwidth, thereby preventing burnout, maintaining deliverable quality, and optimizing team productivity.

